Design and Physical Implementation of a 4-Bit Array Multiplier: RTL to GDSII

Harry Jain (2024112005) Aheb Khan (2026802001)

Abstract

This report presents the RTL design, functional verification, logic synthesis, static timing analysis, and physical implementation of a 4-bit unsigned array multiplier. The design is verified through exhaustive RTL simulation and gate-level simulation, synthesized with Yosys using the SKY130 high-density standard-cell library, analyzed with OpenSTA, and taken through an open-source physical-design flow to generate a routed layout. The report summarizes cell count, area, timing estimates, and physical verification results, including design rule checking (DRC) and layout versus schematic (LVS).

1 Part A: RTL Design and Verification

1.1 Example Multiplication

For the assigned example, the last digit of the selected roll number is 1, Hence $A = 1 + 5 = 6$. Since $A \neq 7$, the second operand is $B = 7$. The expected unsigned product is

$$6 \times 7 = 7, \qquad 0110_2 \times 0111_2 = 00101010_2.$$

Figure 1. shows the worked example, including partial products and their summation.

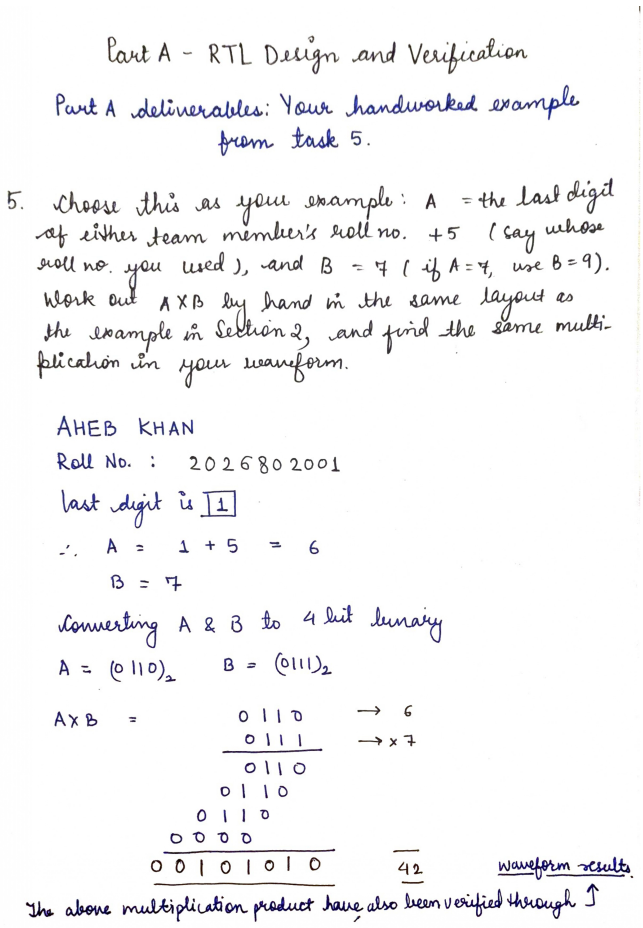


Figure 1: Worked example for the 4-bit unsigned multiplier: $6 \times 7 = 42$.

1.2 Functional Verification

An exhaustive self-checking RTL testbench was used to test all $2^4 \times 2^4 = 256$ combinations of the two 4-bit unsigned inputs. All 256 combinations passed. A separate fault-injection run deliberately introduced an incorrect full-adder result; the testbench detected the incorrect output and reported failure, as expected. Gate-level simulation also passed.

- **RTL functional test:** PASS for all 256 input combinations.
- **Fault-injection test:** The deliberately incorrect full-adder implementation was detected by the testbench.
- **Gate-level simulation:** PASS.

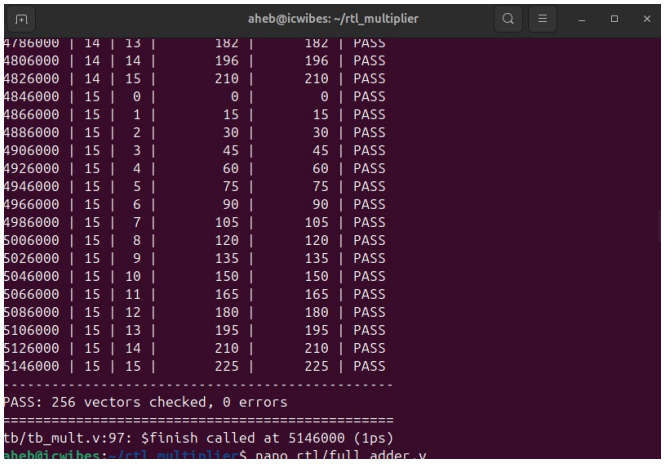


Figure 2: Verification screenshot for the normal passing run.

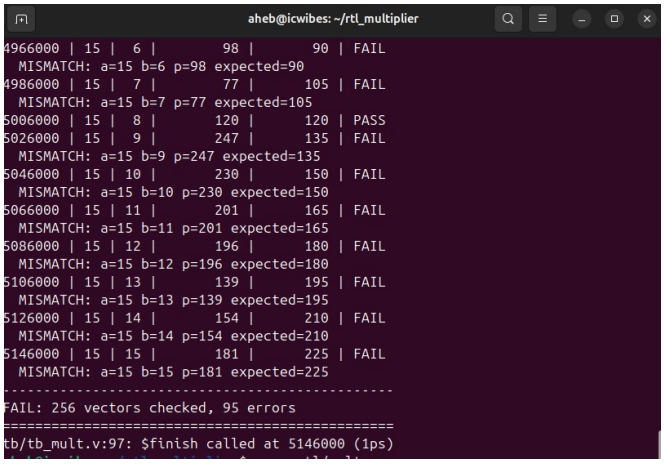


Figure 3: Verification screenshot for the fault-injection run.

1.3 Simulation Waveform

The waveform records the clock, active-low reset, input operands, and registered product output. In the observed waveform, the input is read at a falling clock edge and the corresponding output is available at a rising edge. Measured from the falling-edge input event to the output event, the observed latency is approximately 1.5 clock periods. This measurement convention is stated explicitly because latency measured from a falling edge differs from a latency measured from a rising edge.



Figure 4: Simulation waveform showing input sampling and registered product output.

1.4 Multiplier Architecture

The conceptual 4-bit array multiplier generates 16 partial products using AND operations and combines them with an addition network containing 12 full adders. The registered implementation also contains sequential elements to hold intermediate or output values. Synthesis may optimize this conceptual structure into a different combination of standard cells.

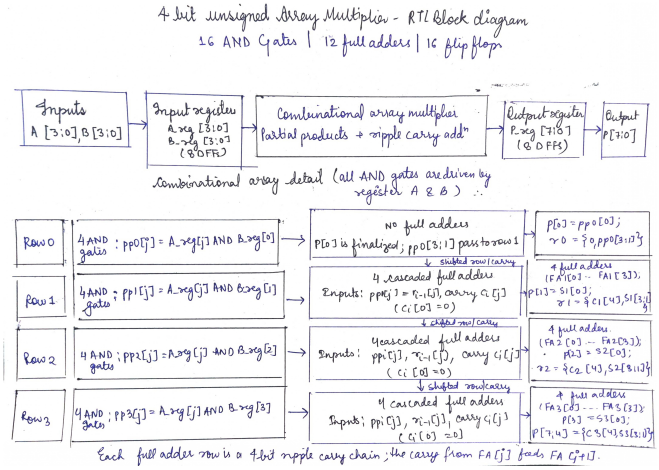


Figure 5: Conceptual multiplier architecture showing partial products, the full-adder network, and pipeline registers.

2 Part B: Logic Synthesis with Yosys

The RTL design was synthesized with Yosys and mapped to the SKY130 high-density standard-cell library. The reported synthesis statistics are summarized in Table 1.

Table 1: Logic synthesis results

Metric	Value
Total mapped cells	76
Flip-flops	16
Flip-flop cell type	`dfrtp_1`
Total cell area (μm^2)	804.522
Flip-flop area (μm^2)	400.384
Flip-flop area (% of total)	49.77%
Most frequent cell types	
`dfrtp_1`	16
`nand2_1`	12
`xnor2_1`	4
Tie at count 4: `nand3_1`, `xnor2_1` *and* `a21oi_1`.	

The number of non-flip-flop cells is

$$N_{\text{comb}} = 76 - 16 = 60.$$

These 60 cells implement the mapped combinational logic. The conceptual architecture's 16 AND gates and 12 full adders are not necessarily preserved as individual standard cells: synthesis optimizes the Boolean logic and maps it to the available library. For example, compound AOI/OAI gates, NAND gates, and XOR/XNOR gates may implement parts of the original arithmetic network.

The flip-flops occupy

$$\frac{400.384}{804.522} \times 100\% \approx 49.77\%$$

of the reported mapped cell area.

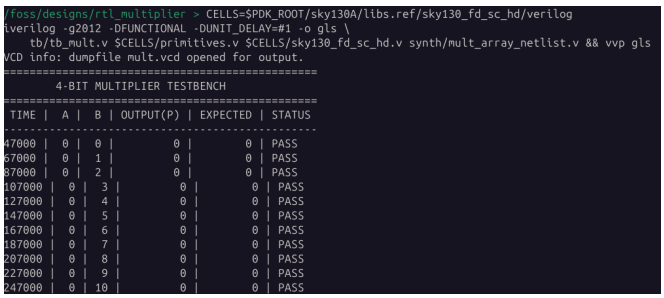


Figure 6: Post-Synthesis gate-level verification result (first captured view).

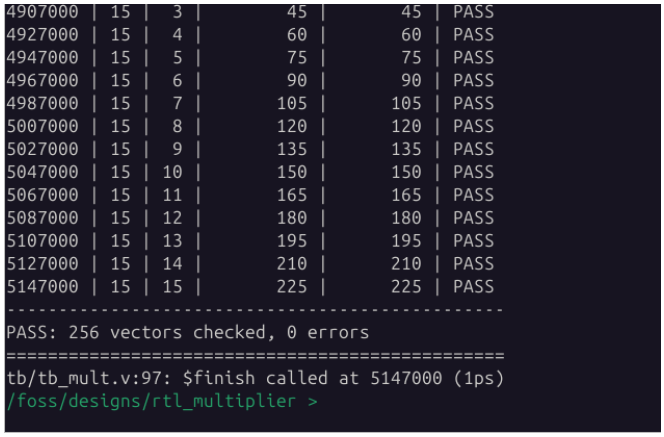


Figure 7: Post-Synthesis gate-level verification result (second captured view).

3 Part C: Static Timing Analysis with OpenSTA

OpenSTA was run using the SKY130 HD typical-corner library (TT, 25°C, 1.80 V). The supplied STA script uses a default clock period of 10 ns, with 1 ns input and output delays. The reported overall worst maximum slack is approximately +6.80 ns. Using the assignment's estimate $T_{\min} = T_{\text{test}} - \text{WNS}$,

$$T_{\min} \approx 10 - 6.80 = 3.20\,\text{ns},$$

and

$$F_{\max} \approx \frac{1}{3.20\,\text{ns}} = 312.5\,\text{MHz}.$$

This is an estimate derived from the reported worst slack and the assignment's calculation method, it was verified by re-running STA at the estimated minimum period and identifying that the worst slack is approximately zero.

Table 2: Static timing analysis results

Metric	Value
Clock period used for STA	10 ns
Overall worst maximum slack	+6.80 ns
Estimated minimum period, $T_{\min}$	3.20 ns
Estimated maximum frequency, $F_{\max}$	312.5 MHz
Slack at $T = T_{\min}$	0
Worst path startpoint	Input port `b[0]`
Worst path endpoint	Flip-flop `_126_`
Mapped logic cells on that path	8
Nominal two-cycle latency at $T_{\min}$	6.40 ns
Estimated throughput at one result/cycle	312.5 M results/s

The maximum-frequency estimate follows

$$F_{\max} = \frac{1}{T_{\min}}.$$

Assuming the design accepts and produces one result per clock cycle once the pipeline is full, its steady-state throughput is approximately $F_{\max}$. A two-cycle latency model would give

$$L_{\text{nominal}} = 2T_{\min} = 2(3.20) = 6.40\,\text{ns}.$$

This nominal calculation is distinct from the waveform measurement in Part A, which measures approximately 1.5 periods from the observed falling-edge input event to the output event.

3.1 Worst-Path Cell Sequence

The reported worst path begins at input port `b[0]` and ends at flip-flop `_126_`. The mapped combinational cell sequence reported for that path is:

$$\texttt{_086_} \rightarrow \texttt{_058_} \rightarrow \texttt{_065_} \rightarrow \texttt{_074_}$$
$$\rightarrow \texttt{_078_} \rightarrow \texttt{_094_} \rightarrow \texttt{_095_} \rightarrow \texttt{_096_}.$$

Table 3: Mapped cells on the input-to-register worst path

Instance	Cell type
`_086_`	`and2_0`
`_058_`	`o21a_1`
`_065_`	`o31a_1`
`_074_`	`o311ai_0`
`_078_`	`o21ai_0`
`_094_`	`xnor3_1`
`_095_`	`maj3_1`
`_096_`	`xnor3_1`

This worst path is an input-to-register path, not a register-to-register path. A separate register-to-register report showed a path from `_120_` to `_112_` with 0.330 ns data arrival and 9.550 ns path slack at a 10 ns period.

```
Startpoint: _120_ (rising edge-triggered flip-flop
    clocked by clk)
Endpoint: _112_ (rising edge-triggered flip-flop
    clocked by clk)
Path Group: clk
Path Type: max

Fanout      Cap     Slew    Delay     Time
    Description
-----------------------------------------------------
                  0.000    0.000    0.000   clock
    clk (rise edge)
                           0.000    0.000   clock
    network delay (ideal)
                  0.000    0.000    0.000 ^ _120_/
    CLK (sky130_fd_sc_hd__dfrtp_1)
     1    0.002    0.036    0.330    0.330 v _120_/Q
     (sky130_fd_sc_hd__dfrtp_1)
                  0.036    0.000    0.330 v _112_/D
     (sky130_fd_sc_hd__dfrtp_1)
                                    0.330   data
    arrival time
-----------------------------------------------------
                  0.000   10.000   10.000   clock
    clk (rise edge)
                           0.000   10.000   clock
    network delay (ideal)
                           0.000   10.000   clock
    reconvergence pessimism
                                   10.000 ^ _112_/
    CLK (sky130_fd_sc_hd__dfrtp_1)
                          -0.121    9.879   library
     setup time
                                    9.879   data
    required time

                                    9.879   data
    required time
                                   -0.330   data
    arrival time
-----------------------------------------------------
                                    9.550   slack (
    MET)
```

Table 4: Physical-design results

Metric	Value
Clock period configured	8.0 ns
Implementation runs reported	1
Die area (μm^2)	22,500
Core area (μm^2)	17,759.5
Mapped cells before P&R	76
Standard-cell instances after P&R	336
Total instances reported	1,892
Filler-cell instances reported	1,556
Tap-cell instances reported	245
Detailed-route wirelength (μm)	2,117
Nominal-corner setup slack	+3.5157 ns
Worst reported setup slack across corners	+0.5720 ns
Nominal-corner post-route $F_{\max}$ estimate	223.1 MHz
DRC errors (Magic / KLayout)	0 / 0
LVS errors	0

The nominal-corner frequency estimate is calculated from the configured 8 ns period and the reported nominal-corner setup slack:

$$T_{\min,\mathrm{nom}} \approx 8 - 3.5157 = 4.4843\,\mathrm{ns},$$

$$F_{\max,\mathrm{nom}} \approx \frac{1}{4.4843\,\mathrm{ns}} \approx 223.0\,\mathrm{MHz}.$$

This estimate uses the nominal-corner slack. The worst reported setup slack across corners is 0.5720 ns, so it should not be substituted into the nominal-corner calculation without also specifying the corresponding corner and constraints. The physical-design and pre-route frequency estimates are not strictly comparable unless their clock definitions, timing constraints, corners, and slack conventions are aligned.

Physical implementation adds instances that are not part of the original synthesized logic. Tap cells connect wells to supply rails; filler cells fill physical gaps in placement rows; clock buffers distribute the clock; and timing-repair or hold buffers may be inserted to address timing constraints. These cells support physical integrity and timing closure, although not every category contributes to the functional Boolean logic.

4 Part D: Physical Design: RTL to GDSII

The design was taken through the LibreLane physical-design flow. The configuration specifies an 8 ns clock period and a 150 μm $\times$ 150 μm die. Table 4 summarizes the supplied metrics.

4.1 Physical Layout

The final layout is documented with a full-die view and a zoomed view of the standard-cell rows and routing.

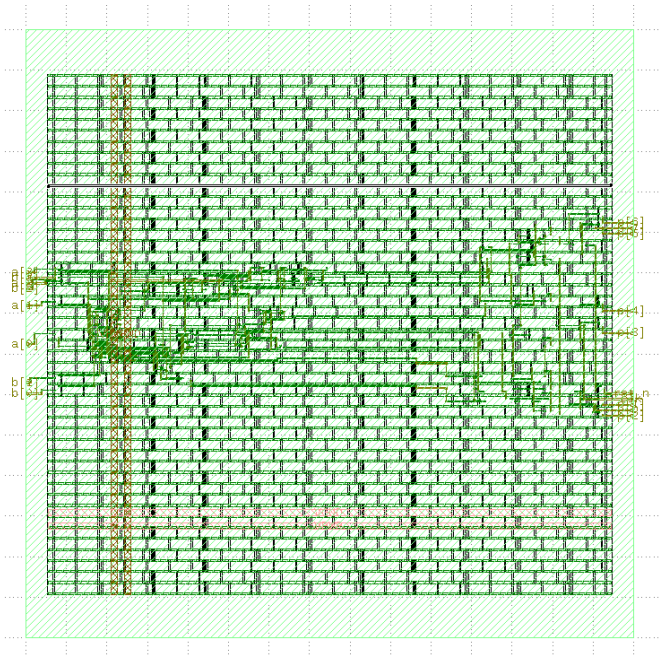


Figure 8: Full-die layout of the 4-bit array multiplier after physical design.

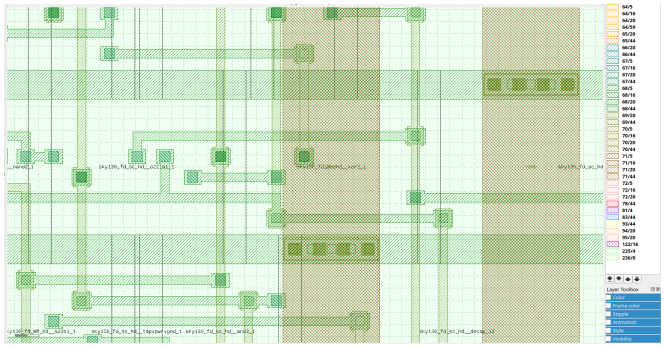


Figure 9: Zoomed view of standard-cell rows and metal interconnects.

4.2 Physical Verification

The supplied metrics report zero Magic DRC errors, zero KLayout DRC errors, and zero LVS errors. DRC checks whether layout geometry obeys the relevant manufacturing rules, including spacing and width rules. LVS compares the electrical connectivity extracted from the layout with the reference netlist. Both are necessary: a layout can satisfy geometric rules but have incorrect connectivity, or be electrically correct while violating manufacturing constraints. Passing these checks is important evidence of layout correctness, but does not by itself guarantee every aspect of functional or reliability sign-off.

5 Contributions

The assignment was completed collaboratively. Both team members contributed equally to the design and implementation workflow, and the work was tracked through the team's GitHub repository.

6 Questions and Answers

Q1. Synthesis Cell Count

Question: The array multiplier requires 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic, excluding flip-flops? Why is the number not simply $16 + 12$?

Answer: The synthesis report contains 76 mapped cells, including 16 flip-flops. Therefore, the number of non-flip-flop cells is

$$N_{\text{logic}} = 76 - 16 = 60.$$

The values 16 AND gates and 12 full adders describe the conceptual architecture, not the final standard-cell count. Synthesis optimizes Boolean equations and maps them to library cells. A full adder can map to multiple standard cells, while compound AOI/OAI cells and Boolean simplification can combine or remove logic.

Q2. Critical Path

Question: What is the critical path of the design, from which register to which register? How many cells are on it, and how does this compare with the estimate of approximately $2N$ full-adder delays?

Answer: The overall worst path reported by the supplied STA run starts at input port `b[0]` and ends at flip-flop `_126_`; it contains eight mapped combinational cells. It is an input-to-register path, not a register-to-register path, so it does not directly answer the register-to-register wording of the question. The separate register-to-register path shown in the available report starts at `_120_` and ends at `_112_`; its data arrival time is 0.330 ns and its slack at a 10 ns period is 9.550 ns. It is not the worst path overall.

For $N = 4$, the rough architectural estimate is

$$2N = 2(4) = 8$$

full-adder delays. The count of eight mapped logic cells on the input-to-register path happens to be numerically equal to this estimate, but the two are not equivalent: mapped cells are not all full adders, and their delays differ.

Q3. Flip-Flop Area

Question: What percentage of the total cell area is occupied by flip-flops? Why is this percentage large for a small design?

Answer: The flip-flop area is $400.384\,\mu\text{m}^2$, and the total mapped cell area is $804.522\,\mu\text{m}^2$. Therefore,

$$\frac{400.384}{804.522} \times 100\% \approx 49.77\%.$$

The design contains 16 flip-flops, so the area cost of the sequential elements is substantial relative to the small combinational datapath. Consequently, flip-flops account for nearly half of the mapped cell area.

Q4. Pre-Route and Post-Route Frequency

Question: Why is the post-route maximum frequency lower than the frequency obtained in Part C? Which cells are added during physical design, and why are they needed?

Answer: The Part C estimate is 312.5 MHz, while the nominal-corner post-route estimate is approximately 223.1 MHz. Routing introduces interconnect resistance and capacitance, and clock-tree synthesis introduces clock insertion delay and skew. Timing repair can also change the buffer structure or cell sizing. The comparison is indicative rather than strictly like-for-like because the OpenSTA run uses a 10 ns default period and the physical-design configuration uses 8 ns; consistent constraints and corners are required for a rigorous comparison.

Examples of additional physical cells include:

- **Filler cells:** Fill placement-row gaps and maintain continuity of physical structures required by the library.
- **Tap cells:** Tie well regions to the appropriate supply rails and help prevent latch-up.
- **Clock buffers:** Distribute the clock while managing transition times and skew.
- **Timing-repair buffers:** Improve slew or delay to help satisfy timing constraints.
- **Hold buffers:** Add delay where minimum-delay checks require it.
- **Decoupling cells, if inserted:** Provide local capacitance to reduce supply-voltage fluctuations.

The metrics show filler, tap, clock-buffer, and timing-repair instances. The exact timing impact of each category should be confirmed from the detailed timing reports.

Q5. DRC and LVS

Question: What do DRC and LVS verify? Why must both pass before a chip can be manufactured?

Answer: Design rule checking (DRC) verifies that the layout obeys the manufacturing rules, including constraints on widths, spacing, and other geometric features. Layout versus schematic (LVS) compares the electrical connectivity extracted from the layout with the reference netlist and checks that the intended devices and connections are present.

Both checks are necessary. A layout may be electrically correct but violate manufacturing rules, or it may satisfy geometric rules while containing incorrect connectivity. Passing DRC and LVS provides important evidence of manufacturability and circuit consistency, but does not alone guarantee all functional, reliability, or sign-off requirements.

Q6. Scaling to an 8-Bit Multiplier

Question: Using the formulas from the assignment, estimate how the number of full adders and critical-path delay would change if the multiplier width increased from 4 bits to 8 bits. Show your working.

Answer: Using the approximate full-adder count

$$N_{\mathrm{FA}} = N(N-1),$$

a 4-bit multiplier requires

$$N_{\mathrm{FA},4} = 4(4-1) = 12,$$

while an 8-bit multiplier requires

$$N_{\mathrm{FA},8} = 8(8-1) = 56.$$

The count therefore increases by a factor of

$$\frac{56}{12} \approx 4.67.$$

Using the rough critical-path estimate of approximately $2N$ full-adder delays,

$$D_4 \approx 2(4) = 8 \text{ full-adder delays}, \qquad D_8 \approx 2(8) = 16 \text{ full-adder delays}.$$

Thus, the estimated logic depth doubles when the width increases from 4 to 8 bits. If the delay of an individual full-adder stage remains approximately constant, the critical-path delay is expected to scale roughly linearly with N. These are architectural estimates; actual area and timing depend on the RTL, synthesis optimization, library, constraints, and physical routing.